Software Design Description

miniaudioengine C++ SDK

Aidan Clyens

Contents

1. Introduction **3**

2. Stakeholders and Design Concerns **4**
2.1. Stakeholders 4
2.2. Design Concerns 4

3. Design Views **6**
3.1. Context View 6
3.2. Composition View 7
3.2.1. SDK Components 7
3.3. Logical View 8
3.3.1. Record Audio/MIDI Input 8
3.3.2. Play Audio/MIDI File 8
3.3.3. Playback to Output Device 9
3.3.4. Audio/MIDI Processing 9
3.3.5. Multiple Tracks 10
3.4. Dependency View 11
3.4.1. Layer Hierarchy 11
3.4.2. External Library Adapters 12
3.5. Information View 13
3.5.1. Audio Session 13
3.5.2. Services 14
3.5.3. Tracks 15
3.5.4. Devices 16
3.5.5. Files 17
3.5.6. Data Plane 18
3.6. Interface View 19
3.6.1. Audio Session 21
3.6.2. Track 21
3.6.3. Device 22
3.6.4. File 23
3.7. Interaction View 24
3.7.1. Control Plane 24
3.7.2. Audio Data Plane 26
3.8. Structure View 28
3.8.1. Library Structure 28

3.8.2. Project Structure 29
3.8.3. Thread Structure 29

4. Design Rationale **31**
4.1. Architectural Design 31
4.1.1. Audio Session 31
4.1.2. Services 33
4.1.3. Adapters 34
4.1.4. Entities 35
4.1.5. Data Plane 36
4.2. External Libraries 37

1. Introduction

2. Stakeholders and Design Concerns

2.1. Stakeholders

Stakeholder	Responsibilities	Design Concerns
Software User	- Monitor audio input device. - Monitor MIDI input device. - Read audio files - Play audio to output device. - Play MIDI to output device.	DC-01 DC-02 DC-03 DC-04 DC-05 DC-06
Third-Party Developer	- Include SDK in C++ software application. - Entity incoming MIDI messages. - Process audio input. - Route processed audio to output device. - Manage audio in multiple tracks.	DC-07 DC-08 DC-09 DC-10 DC-11 DC-12 DC-13 DC-14 DC-17 DC-18
Maintainer	- Maintain CI/CD pipeline. - Manage code repo. - Manage software releases.	DC-13 DC-14 DC-15 DC-16 DC-17
Hardware	- Run on a Windows desktop. - Run on an embedded Linux platform.	DC-13 DC-14

2.2. Design Concerns

ID	Description	Relevant Views
DC-01	Monitor audio input device.	Logical, Interface, Interaction
DC-02	Monitor MIDI input device.	Logical, Interface, Interaction
DC-03	Open and read WAV audio files.	Logical, Information, Interaction

ID	Description	Relevant Views
DC-04	Open and read MIDI files.	Logical, Information
DC-05	Route audio to output device.	Logical, Interface, Interaction
DC-06	Route MIDI to output device.	Logical, Interface
DC-07	Processing incoming MIDI messages.	Logical, Interface
DC-08	Processing incoming audio streams.	Logical, Information, Interface
DC-09	Manage multiple audio tracks.	Logical, Information, Interface, Interaction
DC-10	Add one audio or MIDI input to a track.	Logical, Information, Interface
DC-11	Attach one audio or MIDI output to a track.	Logical, Information, Interface
DC-12	Chain multiple audio processors in one track.	Logical, Information, Interface
DC-13	Build software SDK on Windows and Linux.	Context, Structure
DC-14	Build software SDK on x86_64 and ARM64 platforms.	Context, Structure
DC-15	CI/CD pipeline builds and packages software on all compatible platforms.	Structure
DC-16	Complete unit testing and code coverage.	Structure
DC-17	Package software as an SDK used by third-party software.	Context, Composition, Structure
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.	Composition, Interface, Interaction

3. Design Views

3.1. Context View

Describes the software in context with its external environment. Define users, external components, and the interactions between.

Design Concern	
DC-13	Build software SDK on Windows and Linux.
DC-14	Build software SDK on x86_64 and ARM64 platforms.
DC-17	Package software as an SDK used by third-party software.

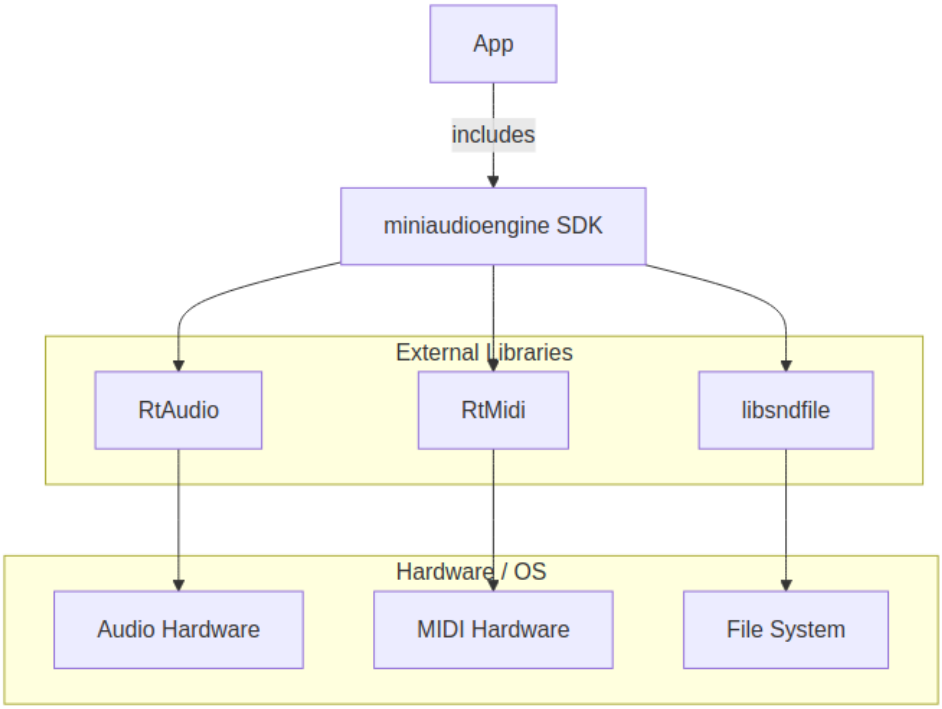

Figure 1: miniaudioengine SDK environment context

3.2. Composition View

3.2.1. SDK Components

Describe the composition of the **miniaudioengine** SDK software libraries.

Design Concern	
DC-17	Package software as an SDK used by third-party software.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

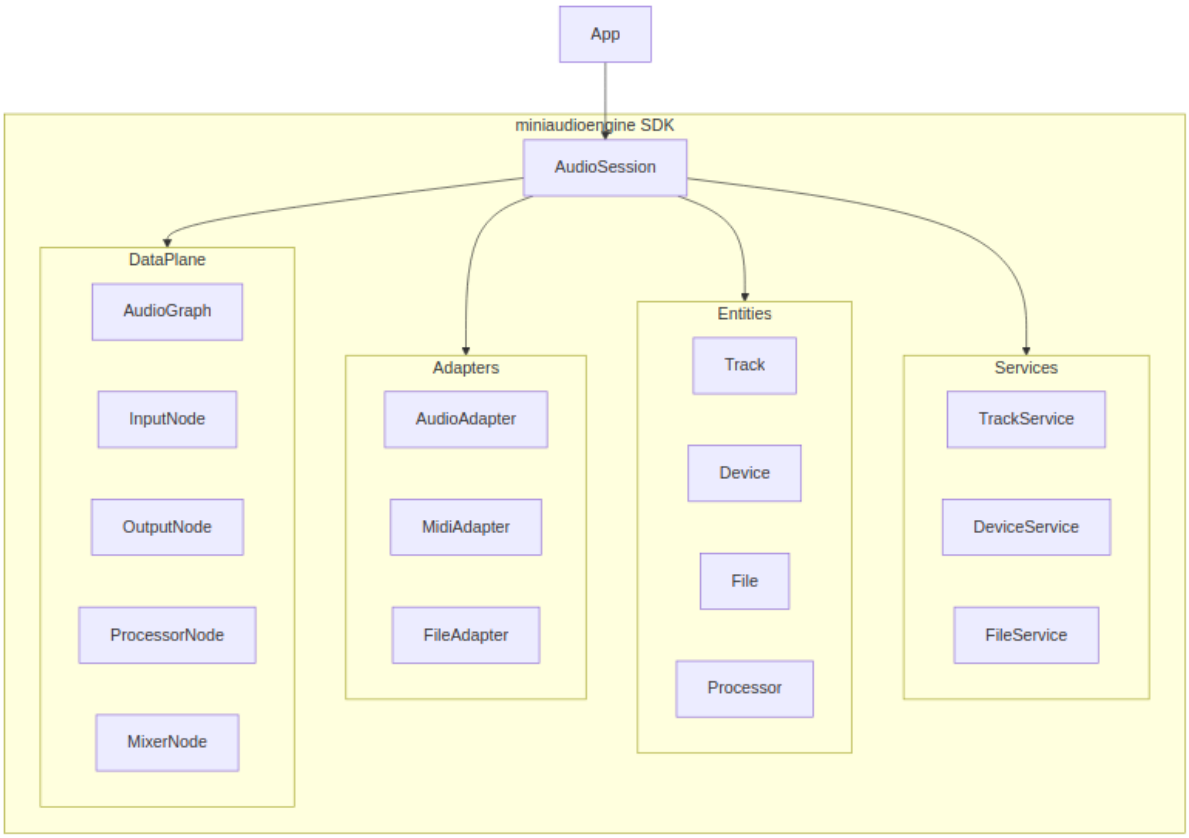

Figure 2: miniaudioengine SDK composition

3.3. Logical View

3.3.1. Record Audio/MIDI Input

Design Concern	
DC-01	Monitor audio input device.
DC-02	Monitor MIDI input device.

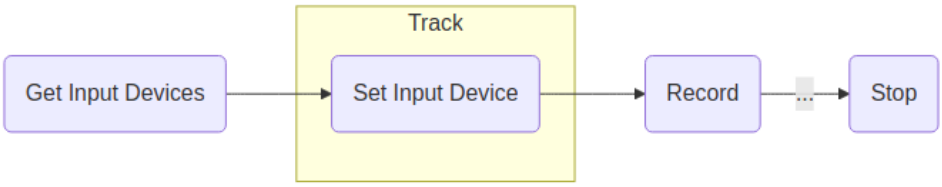

Figure 3: Record Audio/MIDI Input

3.3.2. Play Audio/MIDI File

Design Concern	
DC-03	Open and read WAV audio files.
DC-04	Open and read MIDI files.

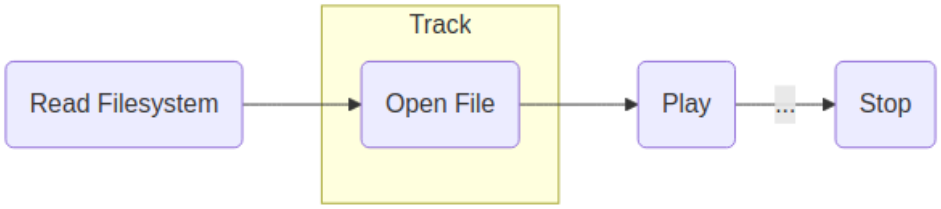

Figure 4: Play Audio/MIDI File

3.3.3. Playback to Output Device

Design Concern	
DC-05	Route audio to output device.
DC-06	Route MIDI to output device.

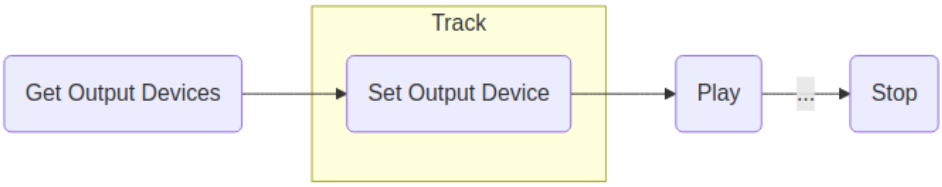

Figure 5: Playback to Output Device

3.3.4. Audio/MIDI Processing

Design Concern	
DC-07	Processing incoming MIDI messages.
DC-08	Processing incoming audio streams.

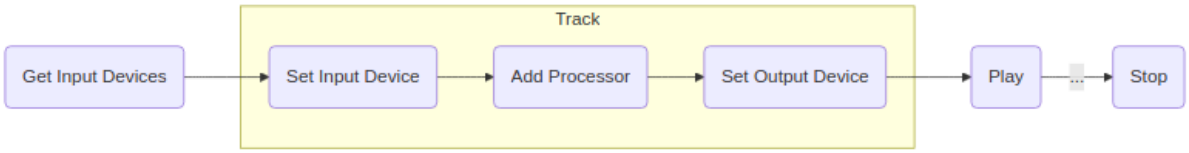

Figure 6: Audio/MIDI Processing

3.3.5. Multiple Tracks

Design Concern	
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.

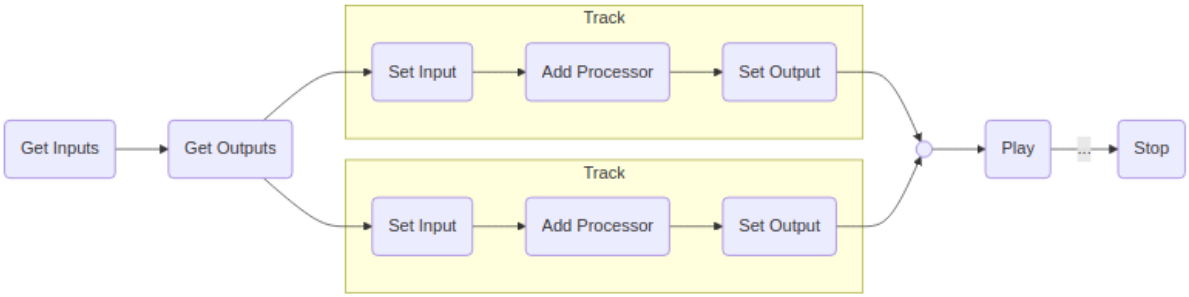

Figure 7: Multiple Tracks

3.4. Dependency View

The components in this SDK depend on the file system, audio and MIDI devices on the host system. Separating devices and files into different services divides the dependency.

3.4.1. Layer Hierarchy

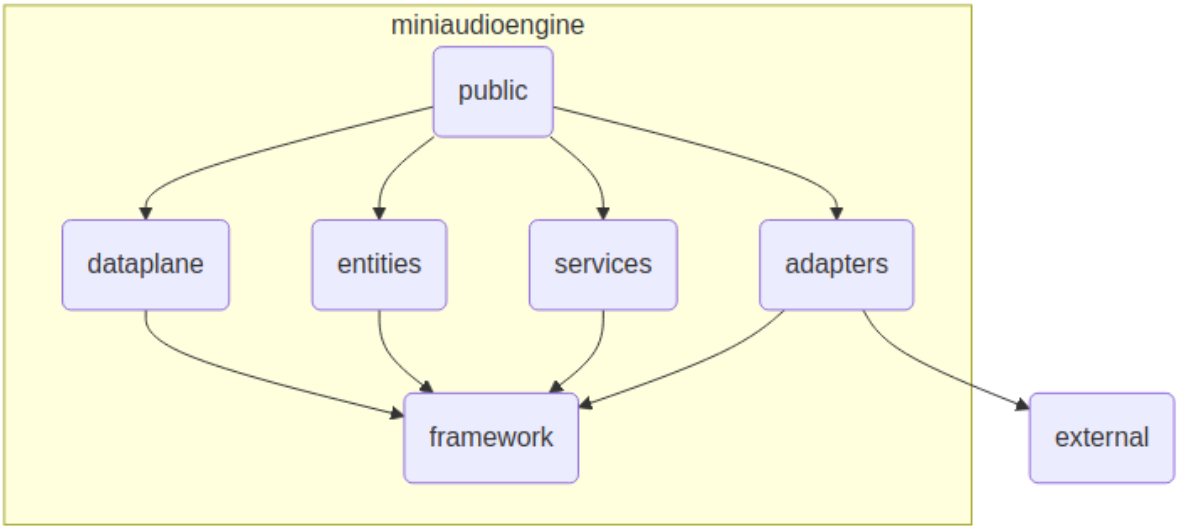

Figure 8: Library Layer Hierarchy

3.4.2. External Library Adapters

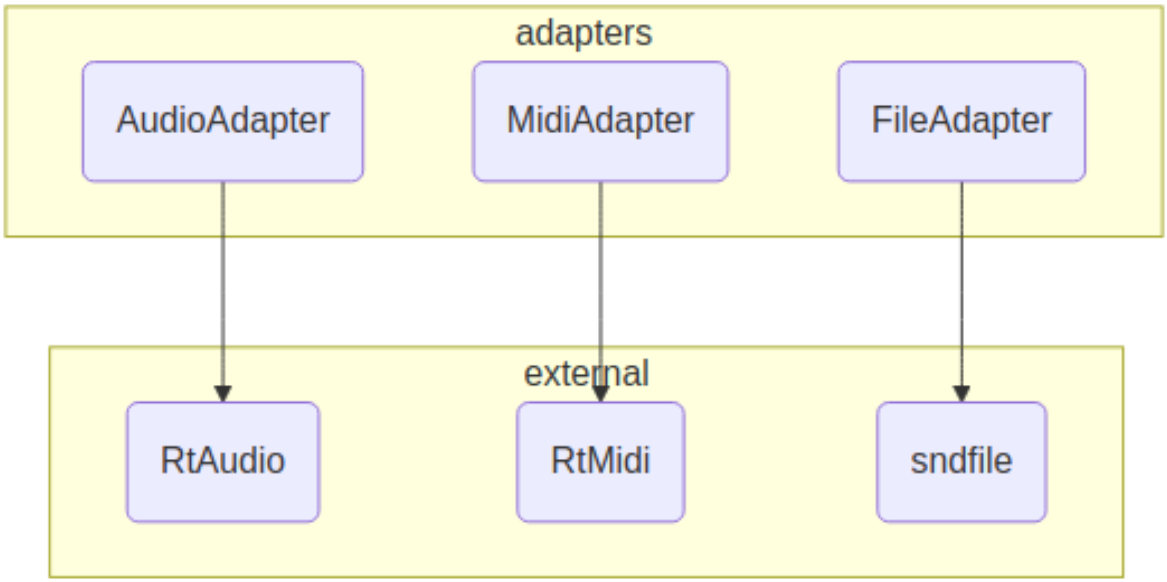

Figure 9: External Library Adapters

3.5. Information View

Design Concern	
DC-03	Open and read WAV audio files.
DC-04	Open and read MIDI files.
DC-08	Processing incoming audio streams.
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.

3.5.1. Audio Session

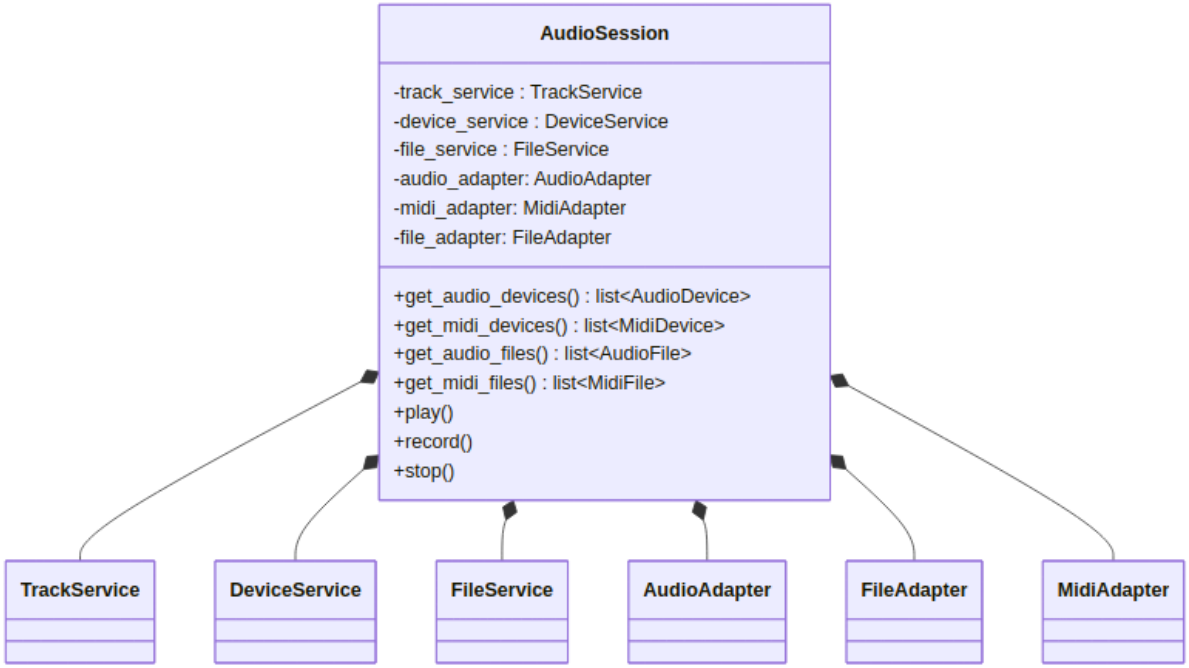

Figure 10: Diagram 10

3.5.2. Services

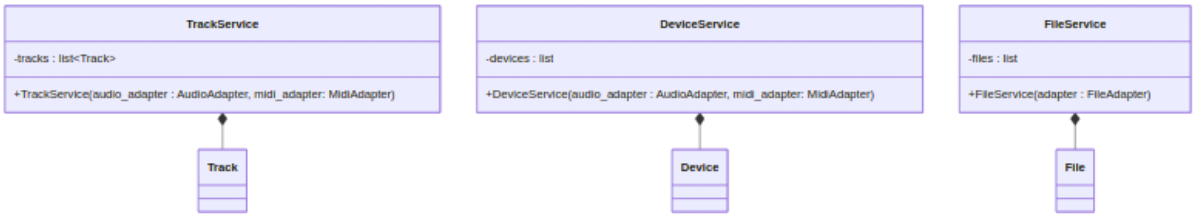

Figure 11: Diagram 11

3.5.3. Tracks

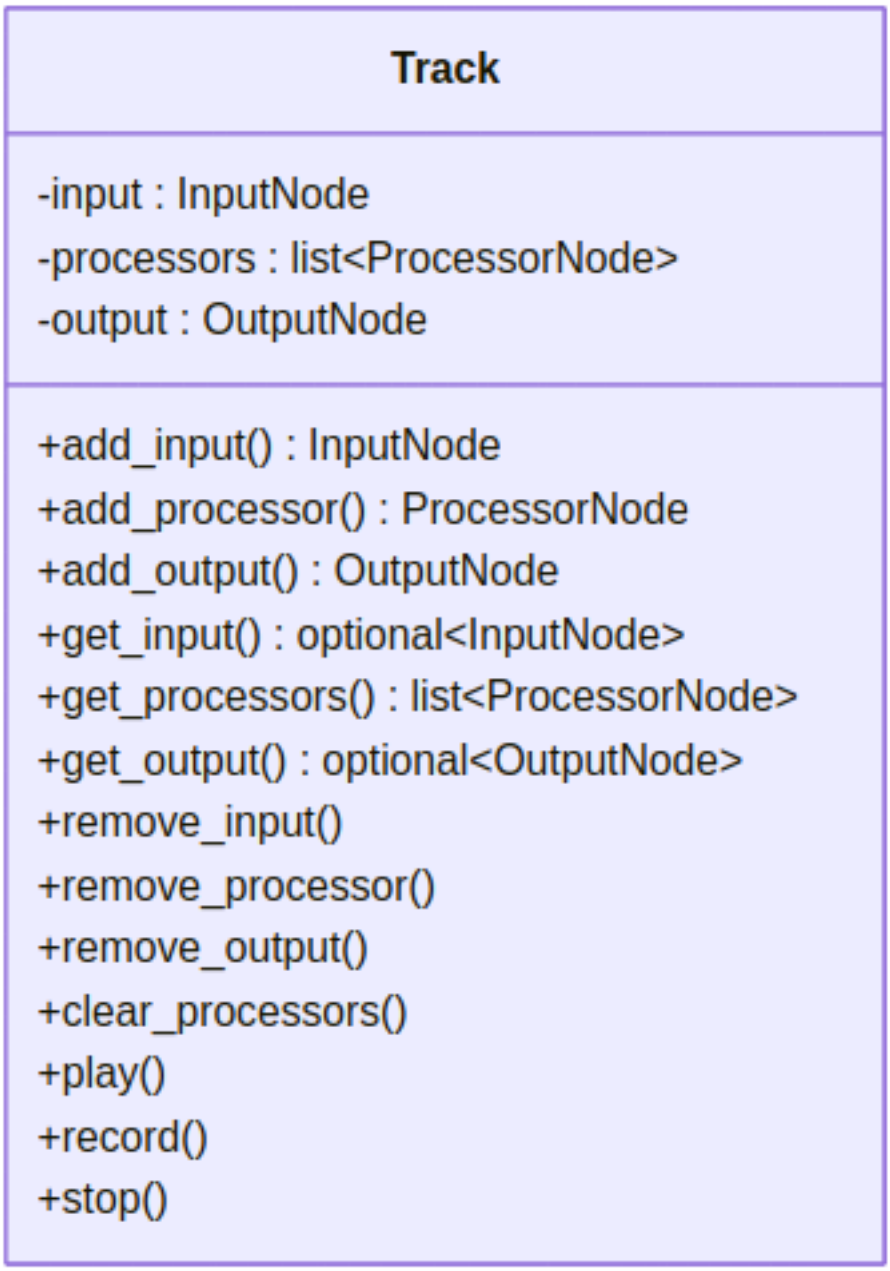

Figure 12: Diagram 12

3.5.4. Devices

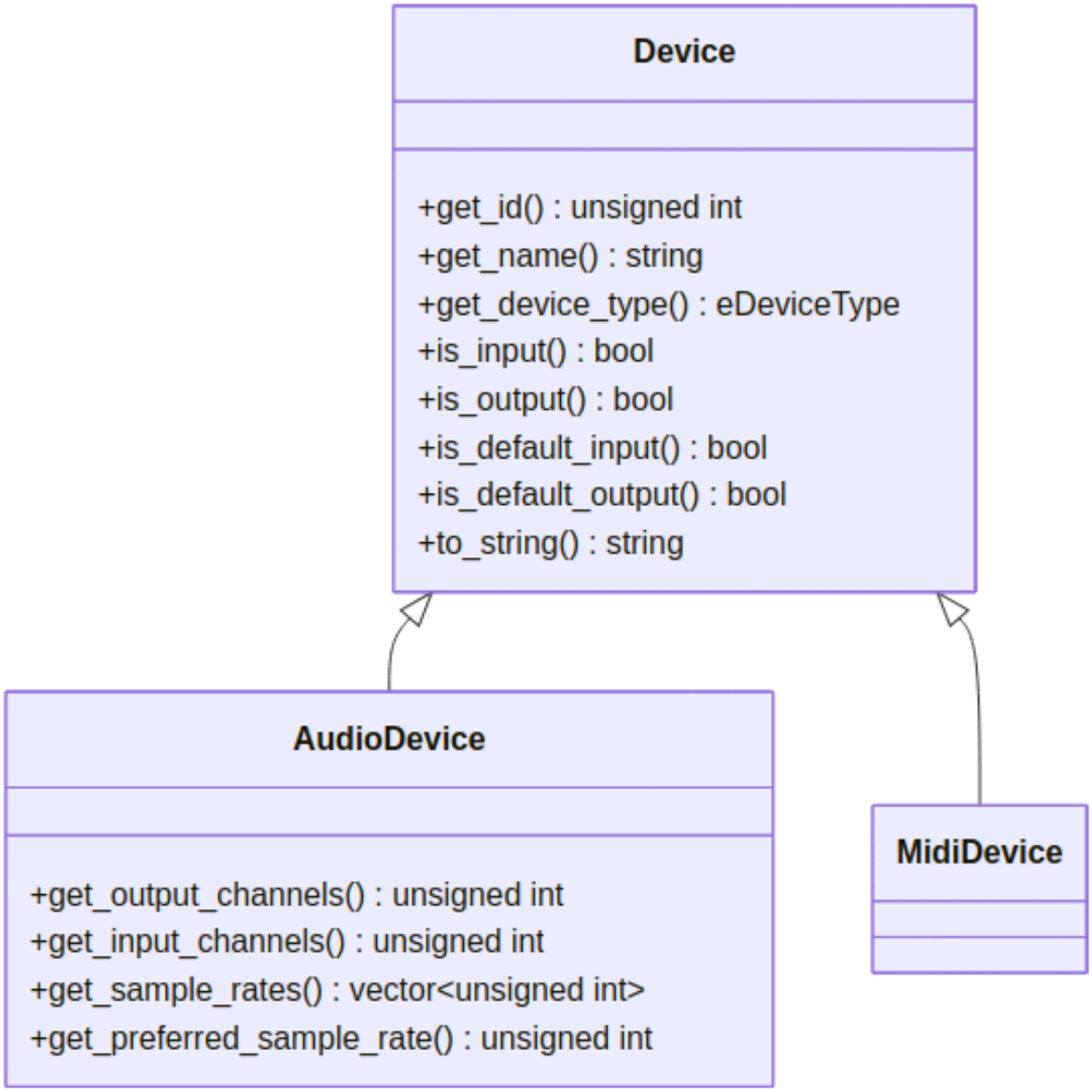

Figure 13: Diagram 13

3.5.5. Files

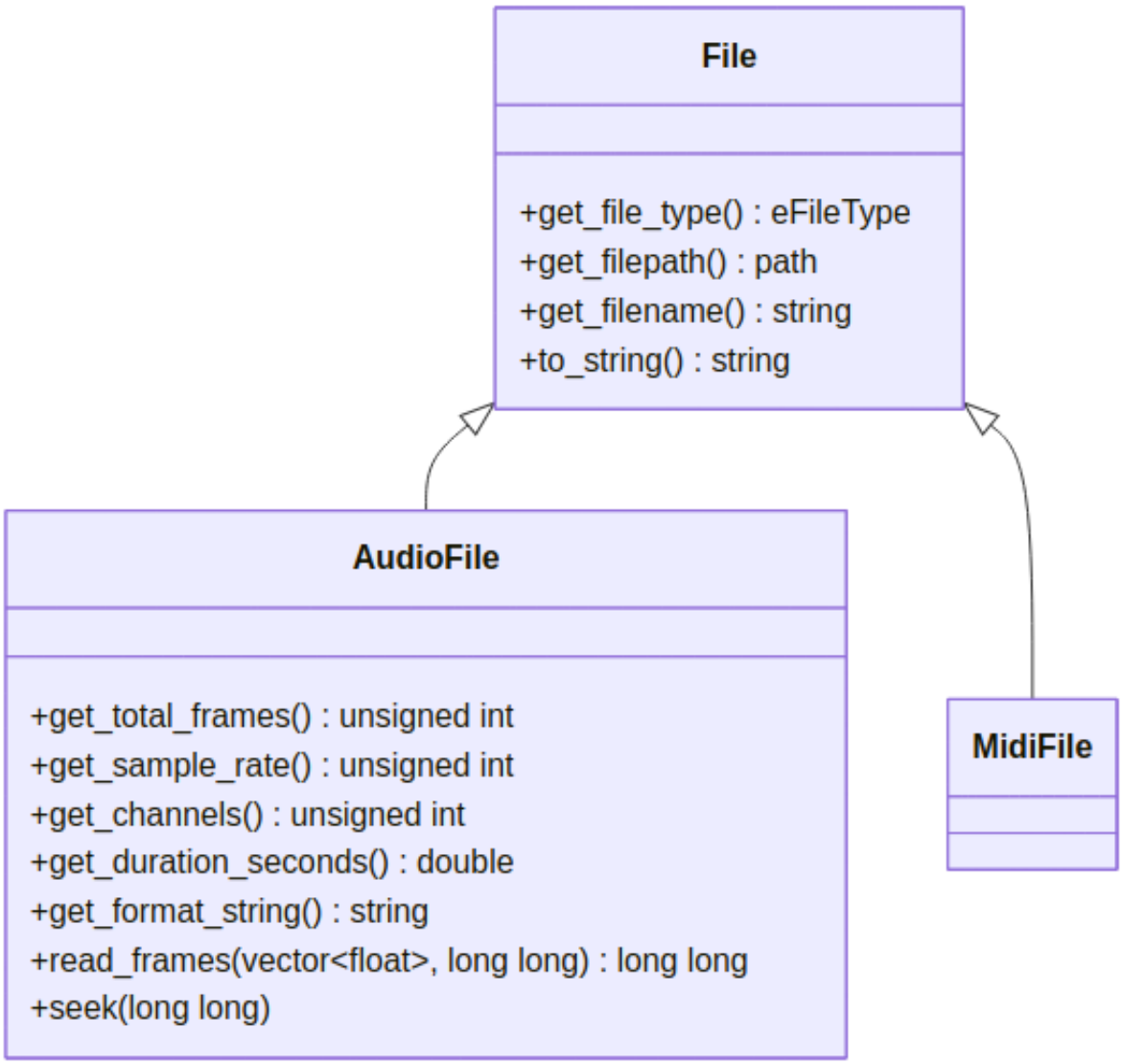

Figure 14: Diagram 14

3.5.6. Data Plane

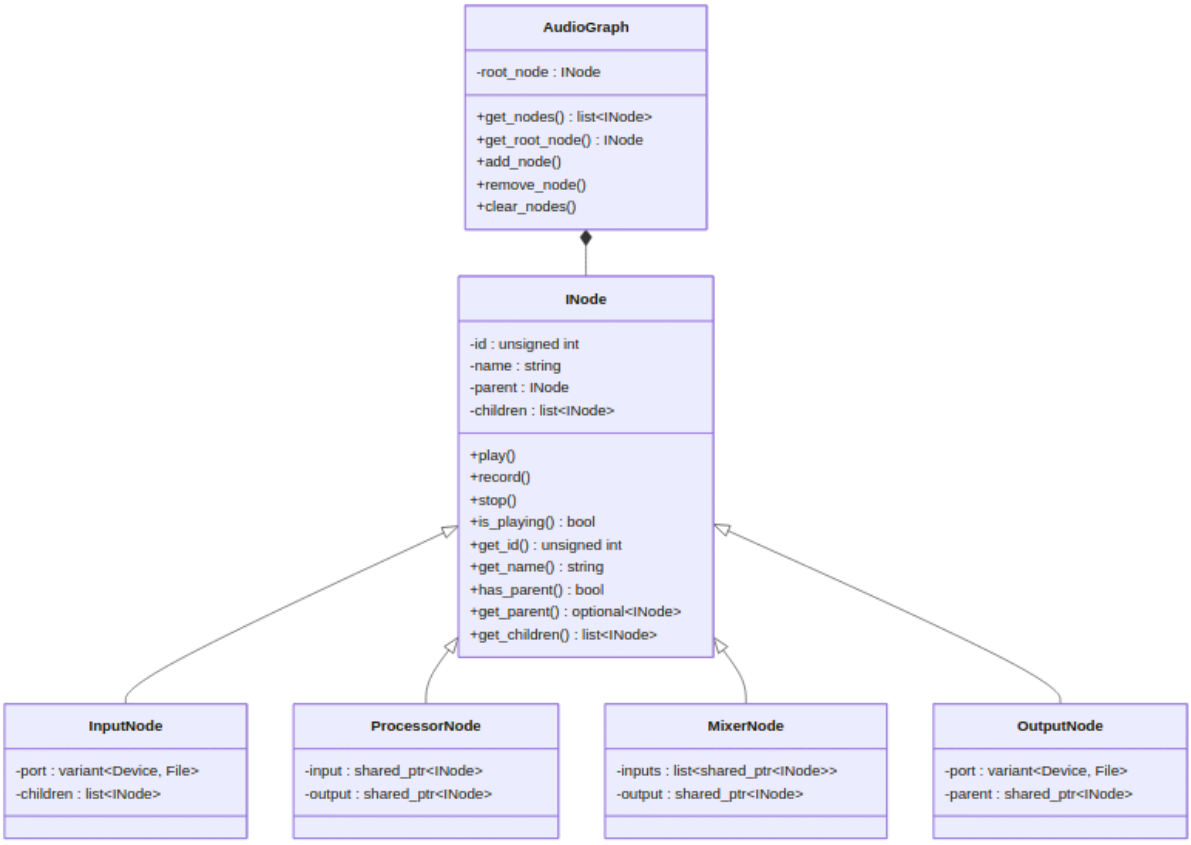

Figure 15: Diagram 15

3.6. Interface View

Design Concern	
DC-01	Monitor audio input device.
DC-02	Monitor MIDI input device.
DC-05	Route audio to output device.
DC-06	Route MIDI to output device.
DC-07	Processing incoming MIDI messages.
DC-08	Processing incoming audio streams.
DC-09	Manage multiple audio tracks.
DC-10	Add one audio or MIDI input to a track.
DC-11	Attach one audio or MIDI output to a track.
DC-12	Chain multiple audio processors in one track.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

The user uses the software via the main `miniaudioengine` SDK library.

The following types need to be accessible to the user:

Component	Description	Location
`AudioSession`		
`Device`		
`File`		
`Track`		
`Processor`		

The following operations need to be accessible to the user:

Operation	Component
Get audio/MIDI devices	`AudioSession`, `Device`
Get audio/MIDI files	`AudioSession`, `File`
Set audio/MIDI device as input or output	`Track`
Set audio/MIDI file as input or output	`Track`
Add a new track	`AudioSession`, `Track`
Add and audio or MIDI processor to a track	`Track`, `Processor`
Start/stop playback	`AudioSession`
Start/stop recording	`AudioSession`

3.6.1. Audio Session

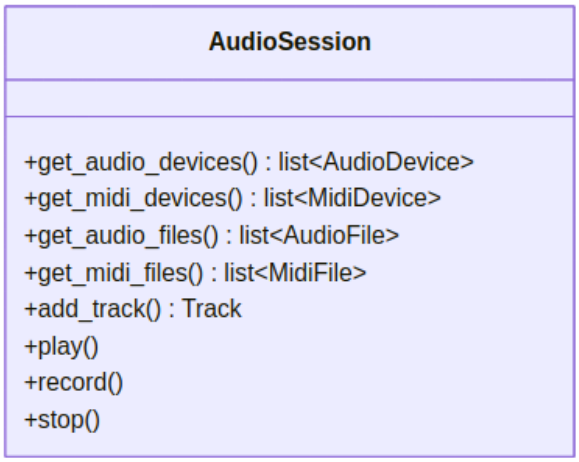

Figure 16: Diagram 16

3.6.2. Track

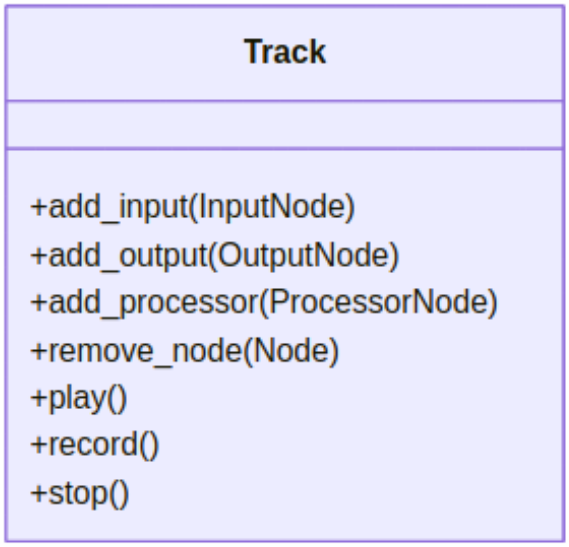

Figure 17: Diagram 17

3.6.3. Device

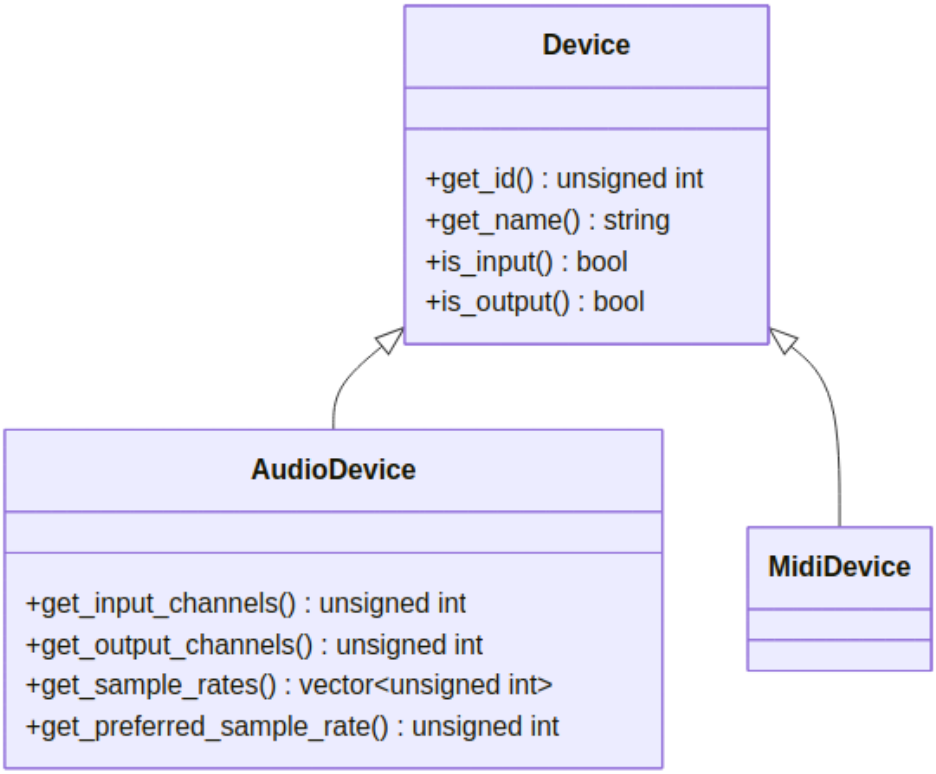

Figure 18: Diagram 18

3.6.4. File

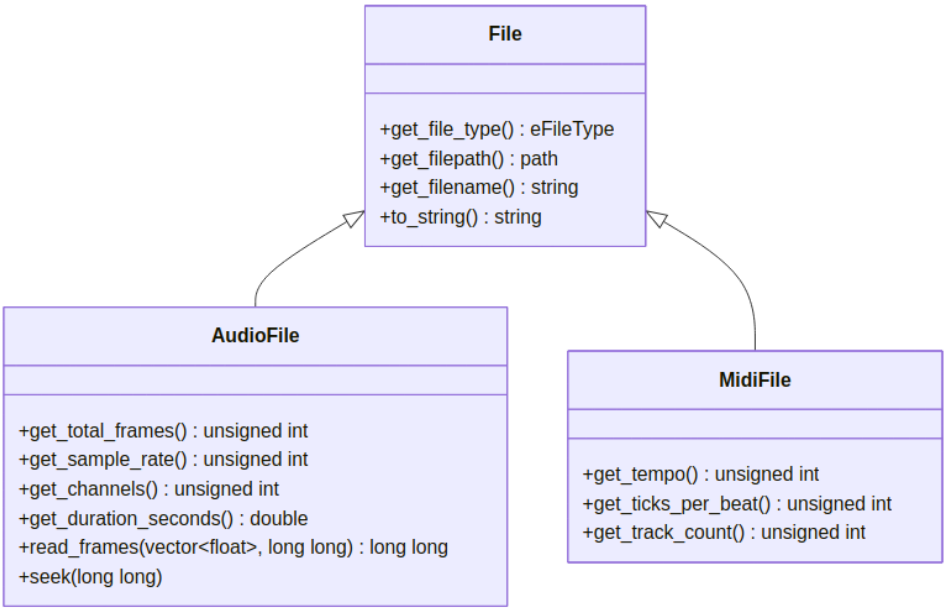

Figure 19: Diagram 19

3.7. Interaction View

Design Concern	
DC-03	Open and read WAV audio files.
DC-05	Route audio to output device.
DC-09	Manage multiple audio tracks.
DC-18	Third-party software developers manage audio tracks, system audio/MIDI devices, and filesystem.

3.7.1. Control Plane

Operation
Get audio/MIDI devices
Get audio/MIDI files
Set audio/MIDI device as input or output
Set audio/MIDI file as input or output
Add a new track
Add and audio or MIDI processor to a track
Start/stop playback
Start/stop recording

e.g. Play Audio File to Output Device

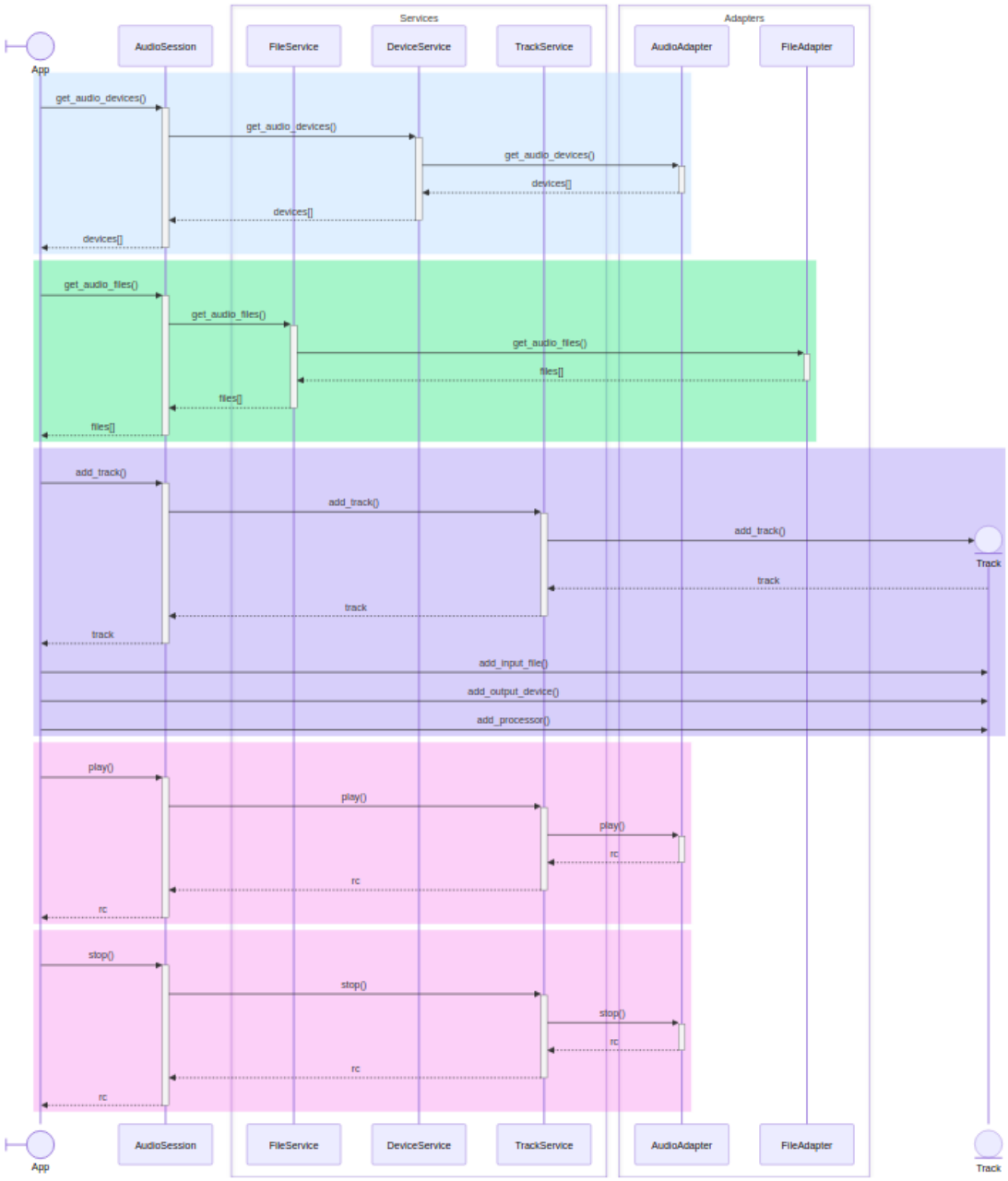

Figure 20: Play Audio File to Output Device

3.7.2. Audio Data Plane

The data plane components are compiled into an acyclic graph as shown below. The `RtAudio` callback a depth-first traversal over the `AudioGraph` to process audio.

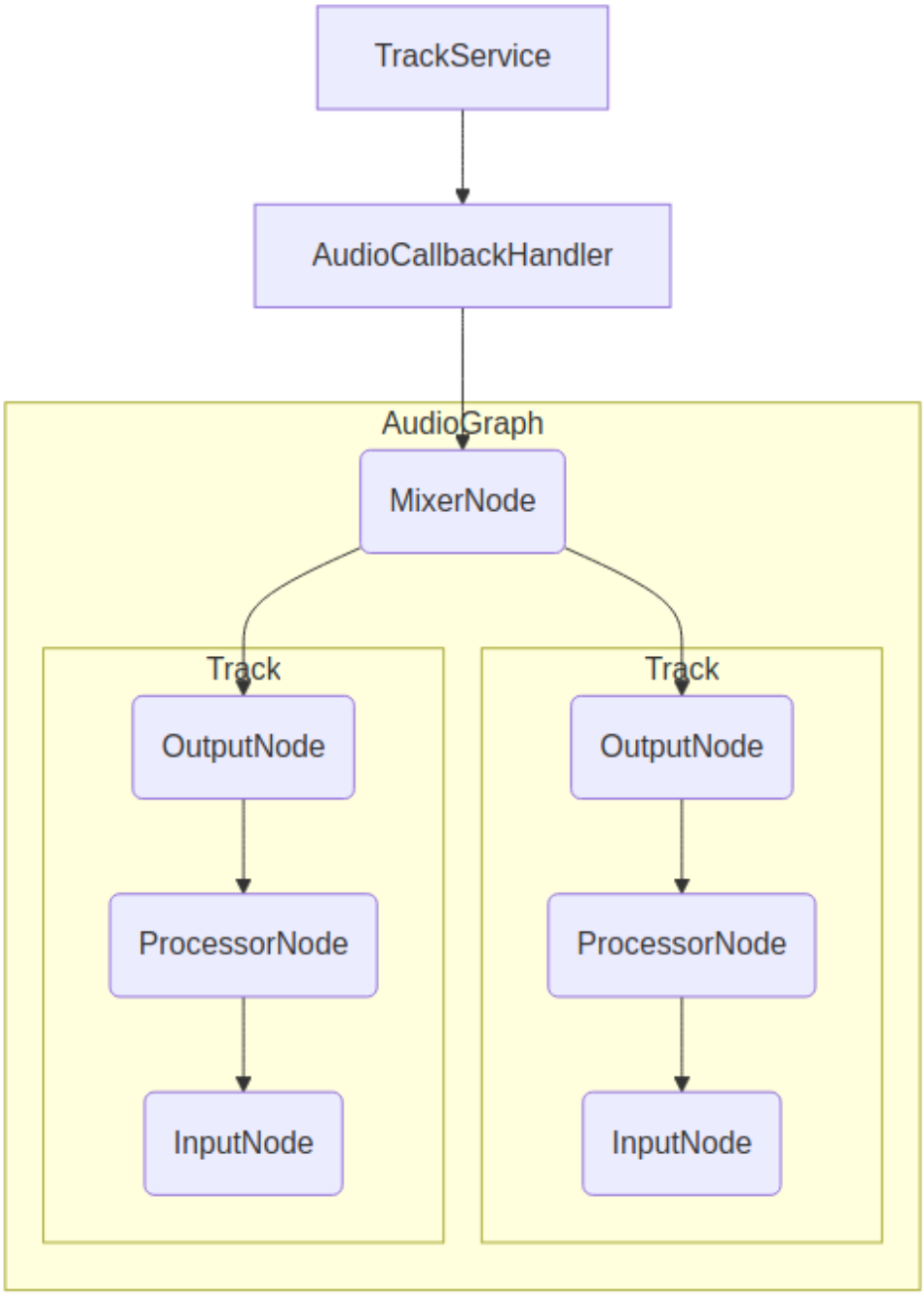

Figure 21: Diagram 21

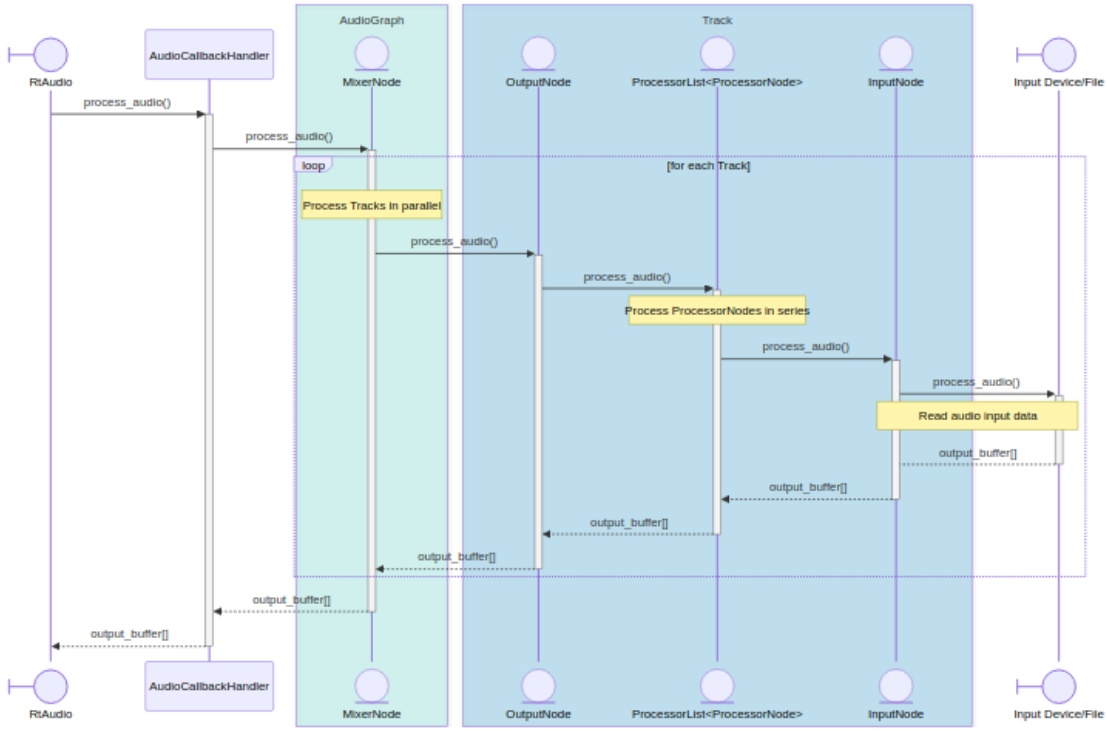

Figure 22: Diagram 22

3.8. Structure View

Design Concern	
DC-13	Build software SDK on Windows and Linux.
DC-14	Build software SDK on x86_64 and ARM64 platforms.
DC-15	CI/CD pipeline builds and packages software on all compatible platforms.
DC-16	Complete unit testing and code coverage.
DC-17	Package software as an SDK used by third-party software.

3.8.1. Library Structure

Internal Library	Components	Namespace
miniaudioengine	`AudioSession`	`miniaudioengine`
entities	`Track`, `Device`, `File`, `Processor`	`miniaudioengine`
services	`TrackService`, `DeviceService`, `FileService`	`miniaudioengine::services`
adapters	`AudioAdapter`, `MidiAdapter`, `FileAdapter`	`miniaudioengine::adapters`
dataplane	`AudioGraph`, `InputNode`, `OutputNode`, `ProcessorNode`, `MixerNode`	`miniaudioengine::dataplane`
framework	`IService`, `IAdapter`, `IEntity`, `INode`	`miniaudioengine::framework`

3.8.2. Project Structure

```
examples/           # Example programs using miniaudioengine SDK
include/
    miniaudioengine/
        audiosession.h          # using namespace miniaudioengine
        track.h
        audiodevice.h
        mididevice.h
        audiofile.h
        midifile.h
        audioprocessor.h
        midiprocessor.h
src/
    framework/
    services/
    entities/
    adapters/
tests/
```

3.8.3. Thread Structure

Thread	Description
Main	Main execution thread.
RtAudio	Real-time audio data processing.
RtMidi	Real-time MIDI message handling.

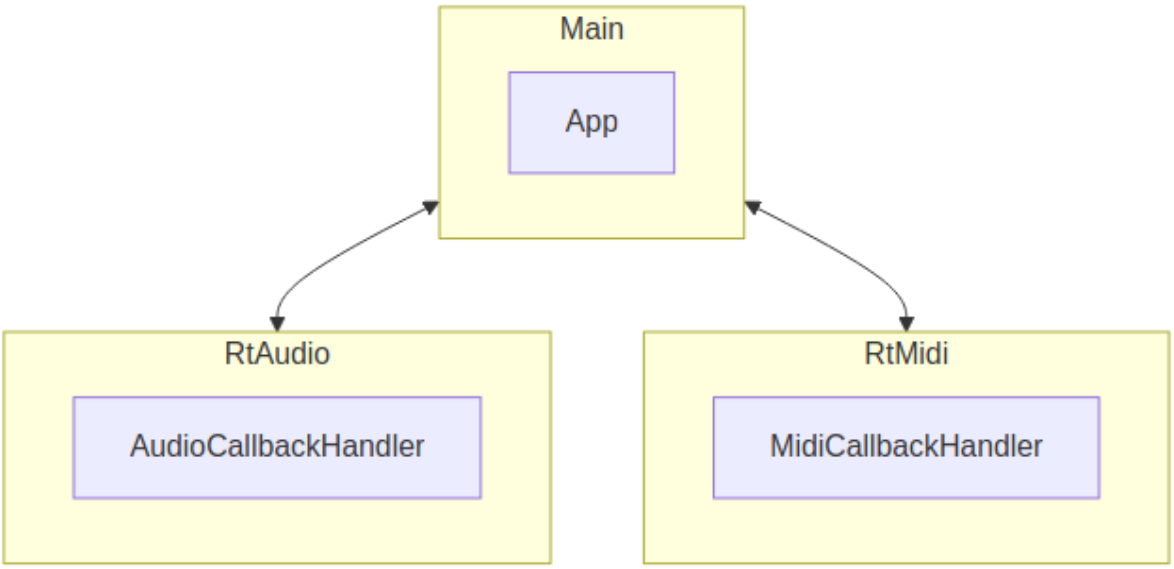

Figure 23: Diagram 23

4. Design Rationale

4.1. Architectural Design

I am keeping the control and data components of this library separate. The main component of the control section is the `AudioSession`. The `AudioSession` owns `Services` for different functionalities. `AudioSession` wraps function calls to the internal `Services` and exposes them to the programmer. `Services` own `Adapters` to interface with external hardware / libraries. `Services` return `Entity` objects for the programmer to interact with.

The programmer can create parallel audio chains using `Tracks` created by the `AudioSession`. `Tracks` have an assigned `Input` and `Output` and can have a list of `Processors` that execute in series.

The data plane is implemented in the `AudioGraph` structure. `AudioGraph` is a tree where the root node is a `MixerNode`. The `MixerNode` may have `OutputNodes`, `ProcessorNodes`, or `InputNodes` as children. `InputNodes` have no children.

4.1.1. Audio Session

This library uses the **Facade** pattern for the `AudioSession` object. `AudioSession` is a wrapper around the internal logic exposed to the programmer. An `AudioSession` owns a `DeviceService`, `FileService`, and `TrackService` that divides and implements the internal engine logic.

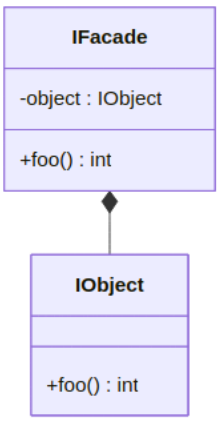

Figure 24: Facade design pattern

```
int IFacade::foo() {
    return object.foo();
}
```

e.g.

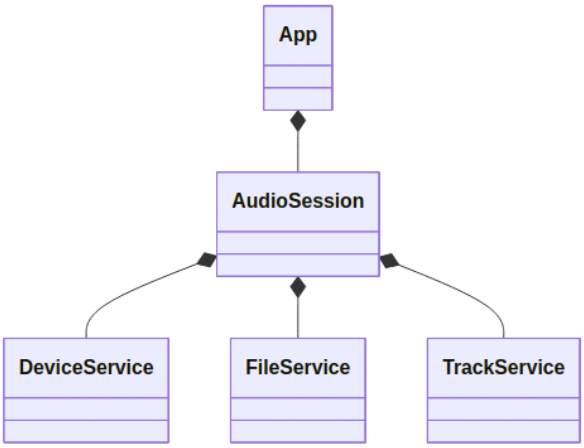

Figure 25: Diagram 25

4.1.2. Services

This library uses the **Factory** pattern for `Services` to create `Entities`.

4.1.3. Adapters

This library uses the **Adapter** pattern to interface with external hardware / libraries. This adds an abstraction layer between the library and the external dependendies, providing isolation of implementation.

4.1.4. Entities

4.1.5. Data Plane

4.2. External Libraries